

AI-Driven Field Work & Task Allocation Engine

Comprehensive Technical Architecture & System Documentation by, Mohammad Raquib Al Hasan

1. Executive Summary & Project Overview

The **AI-Driven Field Work & Task Allocation Engine** is an enterprise-grade, microservice-based platform engineered to optimize the assignment of field service technicians to incoming operational work orders. By leveraging automated machine learning scoring model microservices, the system evaluates technician skill sets, operational experience, and job requirements in real time to deliver optimal task allocation recommendations. The application is designed following decoupled microservice principles, separating concerns across distinct, specialized layers: a modern React frontend dashboard, a robust Node.js/Express core API backend, a Python/FastAPI machine learning engine, a relational MySQL database managed via Prisma ORM, and Docker containerization for local orchestration.

2. System Architecture & Tech Stack Specification

The system comprises four primary execution environments coordinated via Docker Compose locally and distributed across cloud platform providers in production:

Component	Technology Stack	Primary Operational Purpose
Frontend Dashboard	React.js, Vite, Tailwind CSS	Provides an administrative interface for posting work orders, browsing technicians, and reviewing AI match confidence scores.
Core Backend API	Node.js, Express, REST, Prisma ORM	Manages core business logic, handles data persistence, enforces API security, and orchestrates cross-service communication.
Machine Learning Service	Python 3.10+, FastAPI, Scikit-Learn, Uvicorn	Computes suitability scores by comparing job

Component	Technology Stack	Primary Operational Purpose
		requirement vectors against technician profile vectors.
Database Layer	MySQL 8.0, Prisma ORM	Stores transactional application data, user profiles, skill matrices, and historical work order logs.
DevOps & Orchestration	Docker, Docker Compose, Newman CLI	Containerizes application layers for zero-drift local replication and automated integration testing.

3. Database Schema & Data Models

The relational database schema is modeled in MySQL and managed via Prisma ORM. The data structure consists of three core tables enforcing strict relational constraints and indexing:

3.1 Users Table

Stores system entity accounts, including administrators and field technicians.

- **id**: String (UUID v4 / Primary Key)
- **name**: String (Full Name)
- **role**: Enum ('ADMIN', 'TECHNICIAN')
- **email**: String (Unique constraint)
- **location_zip**: String (Postal Code for location matching)
- **createdAt** / **updatedAt**: DateTime timestamps

3.2 Skills Table

Maps field technicians to technical qualifications and experience metrics.

- **id**: String (UUID v4 / Primary Key)
- **user_id**: String (Foreign Key referencing Users.id with CASCADE deletion)
- **skill_name**: String (e.g., 'Cisco Networking', 'React.js', 'Electrical Wiring')
- **experience_years**: Integer (Years of active competency)

3.3 WorkOrders Table

Contains active, assigned, and historical job specifications.

- **id**: String (UUID v4 / Primary Key)
- **title**: String (Short title of the task)
- **description**: Text (Detailed job requirements)
- **status**: Enum ('OPEN', 'ASSIGNED', 'COMPLETED')
- **required_skills**: JSON Array of skill tags
- **assigned_tech_id**: String (Nullable Foreign Key referencing Users.id)

- **createdAt / updatedAt:** DateTime timestamps

4. End-to-End Microservice Data Flow

1. **Job Post / Query:** The administrator interacts with the React Dashboard to inspect an open work order or initiate a technician match query.
2. **Backend Orchestration:** The Node.js Express server receives the request, queries the MySQL database via Prisma to collect open work order requirements and active technician skill profiles.
3. **ML Prediction Request:** Node.js packages the work order requirements and technician vector payload into a JSON request and performs an HTTP POST request to the Python FastAPI microservice (/predict-match).
4. **Machine Learning Scoring:** FastAPI executes the scoring algorithm comparing candidate skills and experience against job requirement weights, returning a probability score matrix.
5. **Response Presentation:** Node.js ranks candidates based on returned scores and sends the formatted candidate list back to the React UI for visual render.

5. Project Directory Structure Blueprint

```

ai-field-work-engine/
├── docker-compose.yml
├── README.md
├── backend-node/
│   ├── Dockerfile
│   ├── package.json
│   ├── prisma7.config.ts
│   ├── src/
│   │   ├── controllers/
│   │   ├── routes/
│   │   ├── services/
│   │   └── server.js
│   └── prisma/
│       ├── schema.prisma
│       └── seed.js
├── ml-python/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── main.py
├── frontend-react/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       ├── components/
│       ├── pages/
│       └── App.jsx
└── tests/
    └── postman_collection.json

```

6. API Endpoint Reference

Service	Endpoint	HTTP Method	Description
Node Backend	/api/health	GET	System health check and operational verification.
Node Backend	/api/work-orders	GET / POST	Retrieve open job lists or post new work orders.
Node Backend	/api/work-orders/:id/match	GET	Fetches AI-allocated technician match probabilities for a specific work order.
Python ML	// /health	GET / HEAD	FastAPI container health check and cold-start wake endpoint.
Python ML	/predict-match	POST	Computes scoring matrix given technician profiles and work order requirements payload.

7. Quality Assurance & Automated Testing

Automated integration testing is configured using Postman collection runner with **Newman CLI**. This enables end-to-end regression testing against live or containerized backend APIs. Execution script configured in backend-node/package.json:

```
npm run test:api
```

The automated suite verifies response codes (200 OK), payload integrity, and JSON response schema structure.

8. Production Cloud Deployment Architecture

The system is deployed using zero-cost cloud hosting providers across decoupled nodes:

- **Frontend Application:** Hosted on Vercel with automatic continuous deployment triggered from the main GitHub branch.
- **Node.js API Service:** Deployed as a Web Service on Render, connected to database connection strings via environment variables.

- **Python ML Microservice:** Deployed on Render with Uvicorn server configuration. Includes HEAD route handler to handle Render ping health checks.
- **Database:** Cloud MySQL instance (Aiven / Render Managed Database) with encrypted network access.

9. Development Roadmap & Summary of Completed Phases

1. **Phase 1:** Root folder initialization, Git repository connection, .gitignore setup, and Docker Compose configuration.
2. **Phase 2:** Node.js/Express backend setup, Prisma ORM installation, database environment configuration, and seed scripts.
3. **Phase 3:** Core REST API routes implementation for user management and work order lifecycle.
4. **Phase 4:** Python FastAPI microservice setup for match prediction algorithms.
5. **Phase 5:** Inter-service communication integration (Node.js API invoking FastAPI ML endpoints).
6. **Phase 6:** React.js (Vite + Tailwind CSS) UI frontend dashboard implementation and full API integration.
7. **Phase 7:** Postman/Newman automated testing suite configuration, system documentation, and final cloud production deployment.